

INFO-F103 – Algorithmique 1

Projet 1 – Backtracking

Mars 2025

1 Introduction

Mancala est le nom donné à une famille de jeux de société qui existerait depuis le début du Moyen-Âge et qui serait apparu en Afrique de l’Ouest¹. Au fil du temps, de nombreuses variantes se sont répandues dans diverses régions du monde – en particulier d’Afrique et d’Asie – créant ainsi une grande diversité de façons de jouer.

Ce projet consiste à réaliser une mini-IA en Python capable de déterminer quel coup jouer au mancala à l’aide du *backtracking* (cf. [Section 4](#)).

2 Règles du jeu

Votre IA se basera sur les règles de l’**Awalé**, une variante du mancala originaire du Ghana.

2.1 Plateau et état initial

Le plateau de jeu consiste en deux rangées de 6 cases dans lesquelles sont réparties 48 graines. Chacune de ces rangées appartient à l’un des deux joueurs.

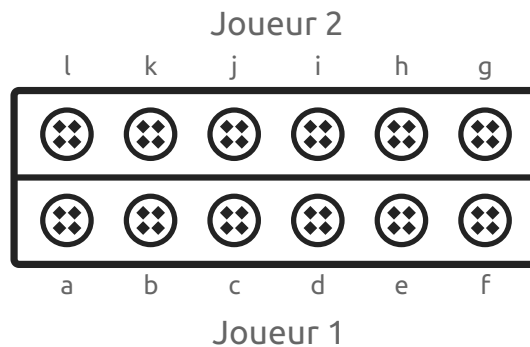


Figure 1: Plateau initial

Au début du jeu, les graines sont équitablement réparties dans les cases : il y a donc 4 graines dans chaque case.

¹ Bien que certaines sources parlent plateaux de jeux retrouvés au Proche-Orient qui dateraient du néolithique.

2.2 Tour de jeu

À chaque tour, les joueurs choisissent une case de leur rangée et prennent toutes les graines de cette case pour les semer une à une dans les cases suivantes dans le sens anti-horaire.

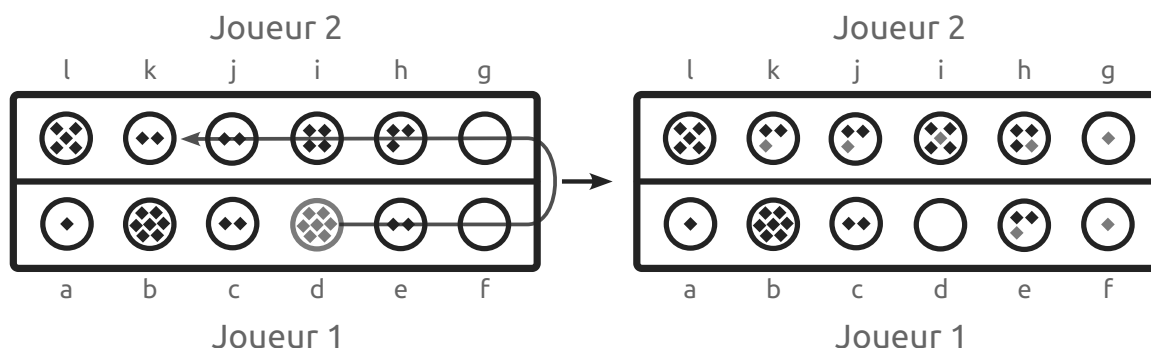


Figure 2: Exemple de coup possible pour le joueur 1

Si la dernière graine tombe dans une case adverse avec 2 ou 3 graines en comptant la dernière déposée, le joueur peut retirer ces graines du plateau et les garder de son côté. Si la case précédente respecte également ces conditions, alors il peut aussi retirer les graines qui s'y trouvent et ainsi de suite jusqu'à ce que les conditions ne soient plus respectées.

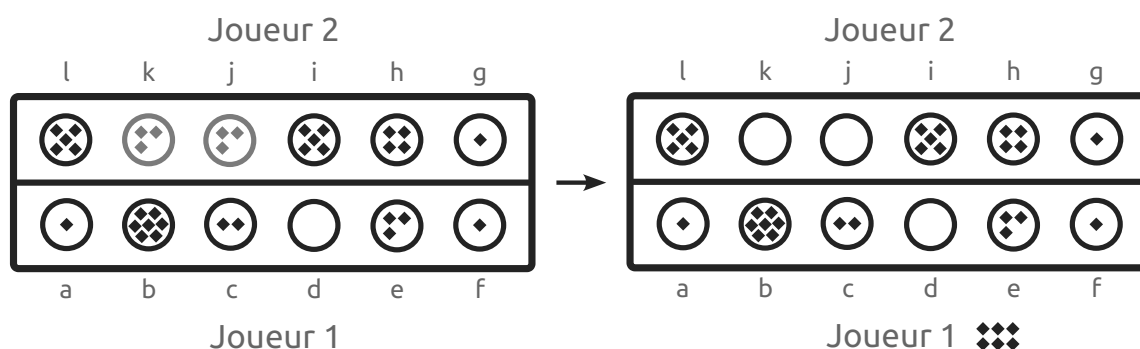


Figure 3: Les deux dernières cases adverses atteintes ont 2 ou 3 graines

Une subtilité intéressante est qu'il est interdit d'*affamer* son adversaire. Un joueur ne peut pas jouer un coup si cela implique que l'autre joueur n'a plus aucune graine dans sa rangée au tour suivant, à moins qu'il n'en soit pas possible autrement.

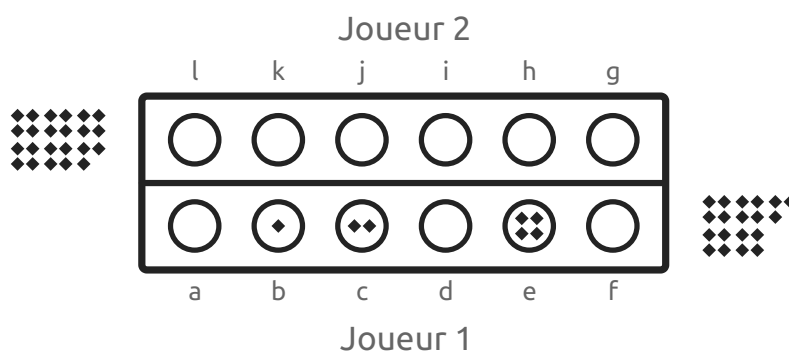


Figure 4: Le coup b est interdit

Dans cet exemple, le coup b est interdit car il affamerait l'adversaire, contrairement au e.

2.3 Fin du jeu

Le jeu s'arrête lorsque l'un des joueurs ne peut plus jouer car il n'y a plus aucune graine dans sa rangée. Le joueur ou la joueuse qui a récolté le plus de graines gagne la partie.

3 Implémentation du jeu

3.1 Plateau de jeu

Le plateau est représenté par une `list` de `list`, où chaque sous-liste correspond à une rangée.

```
board = [  
    [1, 7, 2, 7, 2, 0], # a, b, c, d, e, f  
    [0, 3, 4, 2, 2, 5], # g, h, i, j, k, l  
]
```

La rangée à l'indice 0 est au joueur 1 et celle à l'indice 1 est au joueur 2.

Notez que la deuxième rangée est inversée pour suivre les indices de la [Figure 2](#).

3.2 Tour de jeu

Un tour de jeu peut être joué grâce à la fonction `play`, qui prend comme arguments un plateau, un joueur et une case de ce joueur. Cette fonction modifie le tableau et retourne le nombre de graines récoltées par le joueur à ce tour.

```
def play(board, player: int, cell: int) -> int:  
    # TODO
```

Dans cet exemple, le 2^{ème} joueur joue la case k (sa 5^{ème} case), ce qui lui rapporte n graines :

```
n = play(board, 1, 4)
```

Implémentez cette fonction en tenant compte des explications de la [Section 2.2](#).

3.3 Fin de partie

Implémentez la fonction `is_end`, qui prend un tableau et le joueur dont c'est le tour comme arguments, et retourne un booléen indiquant si la partie est terminée (cf. [Section 2.3](#)).

```
def is_end(board, player: int) -> bool  
    # TODO
```

3.4 Partie

Une partie consiste donc en l'initialisation du tableau `board` et des appels successifs à la fonction `play` jusqu'à ce que `is_end` retourne `True`.

4 IA

Maintenant que les fonctions du jeu en lui-même sont implémentées, vous allez devoir utiliser le *backtracking* pour implémenter une IA très simple basée sur l'algorithme **MinMax**.

4.1 Algorithme MinMax

L'algorithme **MinMax** explore l'ensemble des suites de coups possibles en partant du tour d'un des deux joueurs et sélectionne la meilleure sur base de son score (voir plus bas).

L'exemple ci-dessous montre les coups idéaux sur le plateau de la [Figure 2](#).

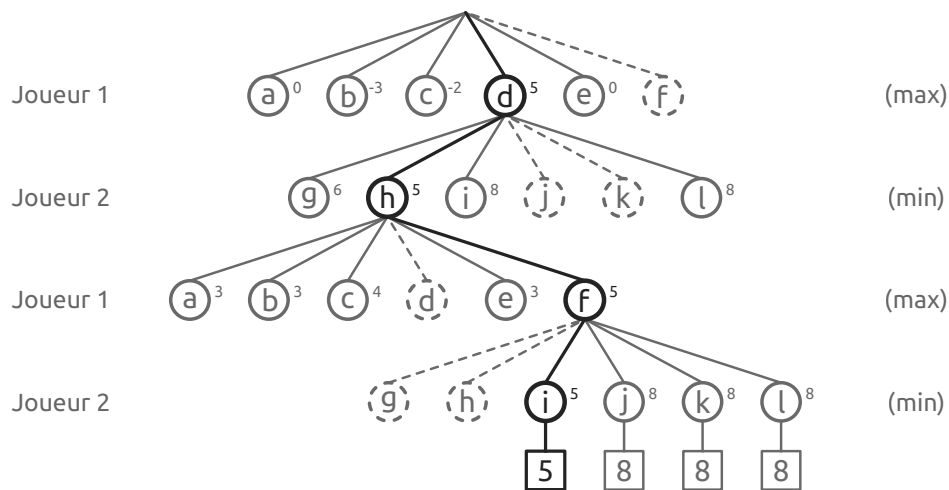


Figure 5: Algorithme MinMax de profondeur 2×2

Chaque étage dans la [Figure 5](#) correspond au tour d'un joueur, qui a le choix entre 6 options. Les coups en pointillés ne sont pas valides car il n'y a aucune graine dans la case.

Le chemin (d,h,f,i) correspond à une suite de coups dont le **score final** est 5.

Score final La différence de graines ramassées par les joueurs lors d'une suite de coups :

$$\text{score final} = \sum \text{graines pour joueur 1} - \sum \text{graines pour joueur 2}$$

Si les joueurs jouent successivement les coups (d,h,f,i), alors le joueur 1 ramassera 5 graines de plus que le joueur 2, ce qui est le meilleur scénario possible étant donné le plateau.

Score Le score de chaque coup correspond au *meilleur* score des coups suivants.

Meilleur score Le joueur 1 cherche à maximiser le score et le joueur 2 à le minimiser.

Pour trouver le meilleur score, MinMax doit essayer tous les coups récursivement. Il choisira le score le plus élevé aux étages du joueur 1 et le plus faible à ceux du joueur 2.

Dans l'exemple de la [Figure 5](#), le joueur 2 choisira la case i pour son deuxième tour, car le score est minimum. Le score du coup f juste au dessus est donc de 5 aussi car le joueur 1 doit supposer que le joueur 2 choisira toujours son meilleur coup. Et ainsi de suite.

4.2 Implémentation

Énumération

Commencez par implémenter une fonction récursive `enum`, qui énumère toutes les suites de coups possibles à l'aide du *backtracking* et retourne une liste de tuples associant une suite de mouvements et le score final pour une profondeur donnée.

```
def enum(board, player: int, depth: int) -> list[tuple[list[int],int]]:  
    # TODO
```

Un exemple d'exécution pour le plateau de la [Figure 2](#):

```
board = [ [1, 7, 2, 7, 2, 0], [0, 3, 4, 2, 2, 5] ]  
enum(board, 0, 4)  
  
# [  
#     ...  
#     ([3, 1, 5, 2], 5),  
#     ...  
# ]
```

Dans cet exemple, les coups successifs (3,1,5,2) ont un score final de 5 (cf. [Figure 5](#)). Les autres éléments ont été masqués, mais la liste contient 596 séquences possibles au total.

Attention: N'oubliez pas d'exclure les coups invalides (cf. [Section 2.2](#)) et d'arrêter la récursion en fin de partie (cf. [Section 2.3](#)).

Vous avez le droit d'ajouter des paramètres supplémentaires tant qu'ils sont optionnels.

Suggestion

Implémentez ensuite la fonction `suggest`, qui détermine quel coup rapportera le plus de graines à court terme sur un plateau donné pour un joueur donné et une profondeur donnée.

```
def suggest(board, player: int, depth: int) -> int:  
    # TODO
```

Utilisez l'algorithme MinMax (cf. [Section 4.1](#)) en vous inspirant de la fonction `enum` pour l'exploration récursive des coups possibles.

Un exemple d'exécution :

```
board = [ [1, 7, 2, 7, 2, 0], [0, 3, 4, 2, 2, 5] ]  
suggest(board, 0, 4) # 3
```

Dans cet exemple, le meilleur coup pour le joueur 1 est la case d car c'est le coup avec le plus haut score, comme on peut le voir sur la [Figure 5](#).

Vous avez le droit d'ajouter des paramètres supplémentaires tant qu'ils sont optionnels. Vous pouvez également créer des fonctions supplémentaires si nécessaire.

5 Tests

Vous retrouverez sur l'UV un fichier de tests à placer dans le même dossier que votre implémentation et qui vous permettra de tester votre projet.

Installez tout d'abord le package pytest avec `pip` :

```
pip install --upgrade pytest
```

Vous pourrez ensuite lancer les tests avec la commande `pytest` :

```
pytest
```

6 Aide

Vous pouvez poser vos questions sur la page UV du cours.

N'hésitez pas non plus à jouer au jeu IRL² pour vous familiariser avec les règles. Vous aurez également une meilleure idée des stratégies que votre *backtracking* devrait sélectionner.

7 Remise

Avant de remettre votre travail, veuillez à bien respecter les points suivants :

1. Le code doit être **commenté**.
2. Le programme doit passer tous les **tests** sous **Python 3.12**.

Consignes de remise

1. Mettez votre code dans un fichier `<matricule>.py`
2. Remise sur l'UV
3. Date limite : le **vendredi 14 mars** avant **22h**

² Vous pouvez utiliser des bouts de papier et des verres, ou n'importe quoi d'autre à votre disposition.